

FEBAUTO Racing

Competitor setup: Linux

Part I Install, clone, first drive

Part II The starter driver, scoring, looking inside

Appendix The starter package, file by file, and how the driver works

Formula Electric at Berkeley

September 2026, platform release v0.1.0

https://github.com/FEBAutonomous/feb-racing

https://febautonomous.github.io/feb-racing/

Contents

1 Install, once 2

2 Clone and set up 2

3 Drive manually 2

4 Drive autonomously with the starter kit 3

5 Score and post 3

6 Looking inside: extra terminals and Foxglove 3

7 When something goes wrong 4

8 Keeping up to date 4

9 What the car gives and takes 5

A The starter package, file by file 5

B How the driver drives, one scan at a time 6

What you are setting up

A simulator of the RoboRacer 1:10 car runs natively on your machine. Your code runs in the box: a Docker container with ROS 2, the bridge to the simulator and the real car's drivers, which keeps what you install and sees your FEB folder. You never install ROS on the machine itself. Foxglove shows you the car's sensors live. `docs/devkit.md` is the one-page reference.

```
~/FEB/                        your folder; the box sees it at /home/feb/FEB
  feb-racing/                 the repo, your clone
    feb                       the launcher
    stack/                    YOUR code (gitignored here: keep it in a repo of your own)
    starter_kit/feb_driver/   the template you copy into stack/
    tracks/                   track folders
~/.feb-sim/app/<os>/          the simulator app, downloaded by setup
Docker                        runs the box
```

The loop: `feb up` builds `stack/` in the box and starts the house nodes; they turn the simulator's stream into the car's topics, `/scan`, `/odom`, `/imu`; your node reads them and publishes `/drive`, a speed and a steering angle; the supervisor passes it to the adapter, the adapter to the simulator; the car moves. On the real car the same node publishes the same topic and the VESC does the rest.

1 Install, once

1. **Docker.** `sudo apt install docker.io`. Then `sudo usermod -aG docker $USER` and log out and back in, so that `docker ps` works without `sudo`. Docker is a background service here; it starts at boot and nothing has to be opened. Check with `docker run -rm hello-world`.
2. **Python 3, git, curl, unzip.** `sudo apt install python3 git curl unzip`.
3. **GitHub CLI.** On Ubuntu 24.04, `sudo apt install gh`. On 22.04 the packaged version is too old: follow the three lines at https://github.com/cli/cli/blob/trunk/docs/install_linux.md. Then `gh auth login`.
4. **Foxglove.** Download the `.deb` from https://foxglove.dev/download and `sudo apt install ./foxglove-studio-*.deb`; open it once, sign in with a free account.

The simulator needs Vulkan. Intel and AMD GPUs work with the stock Mesa drivers; NVIDIA needs the proprietary driver (`sudo ubuntu-drivers install`). Run from a desktop session, not over plain SSH, so the app has a display.

2 Clone and set up

```
mkdir ~/FEB && cd ~/FEB
git clone https://github.com/FEBAutonomous/feb-racing
cd feb-racing
./feb setup --name "Your Name"
```

Pulls the image, creates your box and downloads the Linux build into `~/.feb-sim/app/`. A few minutes. If `./feb up` later fails with a Vulkan error, the GPU driver is the thing to fix; `vulkaninfo -summary` (package `vulkan-tools`) shows what is found.

3 Drive manually

```
./feb up
```

Arrow keys drive. Menu top-left: Manual/Autonomous, Track, Cars, Look, camera. Close the window to stop. Nothing drives by itself without a stack.

4 Drive autonomously with the starter kit

```
cp -r starter_kit/feb_driver stack/
./feb up --stack "ros2 launch feb_driver drive.launch.py"
```

The box builds the package, the simulator opens, the car drives, Foxglove opens on its topics.

What the starter driver is. A complete ROS 2 package. The driver is one commented file, `stack/feb_driver/feb_driver/reactive_driver.py`, a "follow the gap" controller: clean the lidar scan, blank a safety bubble around the nearest obstacle, aim at the widest gap, pick a speed from the room ahead, publish the speed and steering angle on `/drive`. Its numbers are in `stack/feb_driver/config/driver.yaml`.

Editing code. `stack/` is a plain folder on your machine; open it in any editor, or `./feb code` opens VS Code attached to the box, where every terminal is already inside it. Edit, save, run the same `./feb up` command again; the box rebuilds your packages in seconds, and a Python edit needs no rebuild at all.

5 Score and post

```
./feb practice --stack "ros2 launch feb_driver drive.launch.py"
./feb submit
```

`practice` runs one attempt the way a competition does: headless simulator, a warm-up lap plus ten timed laps, 10 s per collision, 300 s limit; it writes `runs/<id>/result.json` and a bag. `submit` opens a pull request with that file; a check validates and merges it, and your time appears on the site's practice board as *unverified*. The organiser's nightly rerun of your registered image on the same track turns it *verified*.

6 Looking inside: extra terminals and Foxglove

More terminals in the box

The box is always there. Open as many terminal windows as you like and, in each, from the repo folder:

```
./feb shell
```

That is a ROS 2 shell inside the box, with the environment already sourced. Useful commands there:

```
ros2 topic list                       # everything on the graph
ros2 topic hz /scan                   # is the sensor stream alive, at what rate
ros2 topic echo /drive                # what your node is sending
ros2 topic echo /feb/state            # what the supervisor is doing with it
ros2 node list                        # your node should be here
ros2 param list /reactive_driver      # live parameters
ros2 bag record -a -o ~/FEB/bags/mybag   # lands on your laptop
```

`exit` leaves the shell; the box keeps running.

The pieces on their own. `feb up` starts the house nodes, the simulator and Foxglove together because that is what you want most of the time, but they are independent. The house nodes alone, for replaying bags or tests: `./feb up -no-sim`, or in a box terminal `ros2 launch feb_tools up.launch.py`. The simulator alone is the app in `~/.feb-sim/app/linux`, keyboard driving, no ROS; it connects to the box by itself whenever the house nodes are up.

Foxglove

Installed in the install list; sign in once. After that, `./feb up` opens Foxglove connected to your car a few seconds after the simulator window. If it does not, or you want a second window: in Foxglove choose *Open connection*, *Foxglove WebSocket*, address `ws://localhost:8765`.

Panels worth adding (the + at the top right of a panel):

- **3D**: add the topic `/scan`; display frame `laser`. The scan appears as points around the car.
- **Image**: `/camera/color/image_raw`.
- **Plot**: `/drive.drive.speed` and `/odom.twist.twist.linear.x` against time: what you asked for and what you got.
- **Raw Messages**: any topic, to read exact values.
- **Topics** (left sidebar): every topic with its rate; a dash means nothing is publishing.

Save the layout (*Layout*, *Save*) and it comes back next time. Foxglove also opens the bags that `practice` records under `runs/<id>/`, and any bag you record yourself: *Open local file*.

7 When something goes wrong

```
./feb logs -n 100          # build errors, your node's output
./feb shell                # a ROS 2 terminal inside the box
./feb reset                # a fresh box; your files are untouched
```

Nothing drives: correct without `-stack`; with one, read `./feb logs`. Port 4567 in use: `./feb reset`. Docker pull denied, or "cannot connect to the Docker daemon": the Docker service is stopped (`sudo systemctl start docker`) or you are not in the `docker` group yet.

8 Keeping up to date

```
./feb update
```

One command pulls the repo (tracks, launcher, docs, new starter packages), pulls a newer image if there is one and replaces the box with it (what you had installed by hand in the box goes, your files do not), and replaces the simulator app when a new build is released. Your `stack/` folder is never touched. When a new starter package appears in `starter_kit/`, copy it the same way as the first one. The cone tracks (`loop_cones`, `spielberg_cones`) have their own baseline, the cone driver:

```
cp -r starter_kit/feb_cone_driver stack/
./feb up --track loop_cones --look visual --stack "ros2 launch feb_cone_driver drive.
    launch.py"
```

It needs the Visual look: the camera's colour thresholds are tuned for the dressed scene. `docs/starter-kit.md` describes what it does. Whatever stack you run, `./feb logs -f` in a second terminal shows what `feb up` printed: the package build, the house nodes, and every line your nodes print.

9 What the car gives and takes

Topic	Meaning
`/scan`	`LaserScan`, 1080 beams over 270°, 0.06 to 10 m, frame `laser`
`/odom`	`Odometry`: pose and speed dead-reckoned from the wheels and the gyro, as the car's VESC does it
`/imu`	orientation, angular velocity, linear acceleration
`/camera/color/image_raw`	the front camera
`/drive`	**you publish this**: `AckermannDriveStamped`, `drive.speed` in m/s, `drive.steering_angle` in rad, at most ±0.5236
`/feb/state`	the supervisor: IDLE, MANUAL, AUTONOMOUS or STOP, and why
`/autodrive/roboracer_1/*`	the simulator's own topics: ground truth, lap and collision counters. Debugging only, barred at race time; the car does not have them

Publish as many topics of your own as you like; only `/drive` reaches the car. Use message stamps or `use_sim_time`, never the wall clock, for speeds and integrals. A command older than half a second stops the car.

A The starter package, file by file

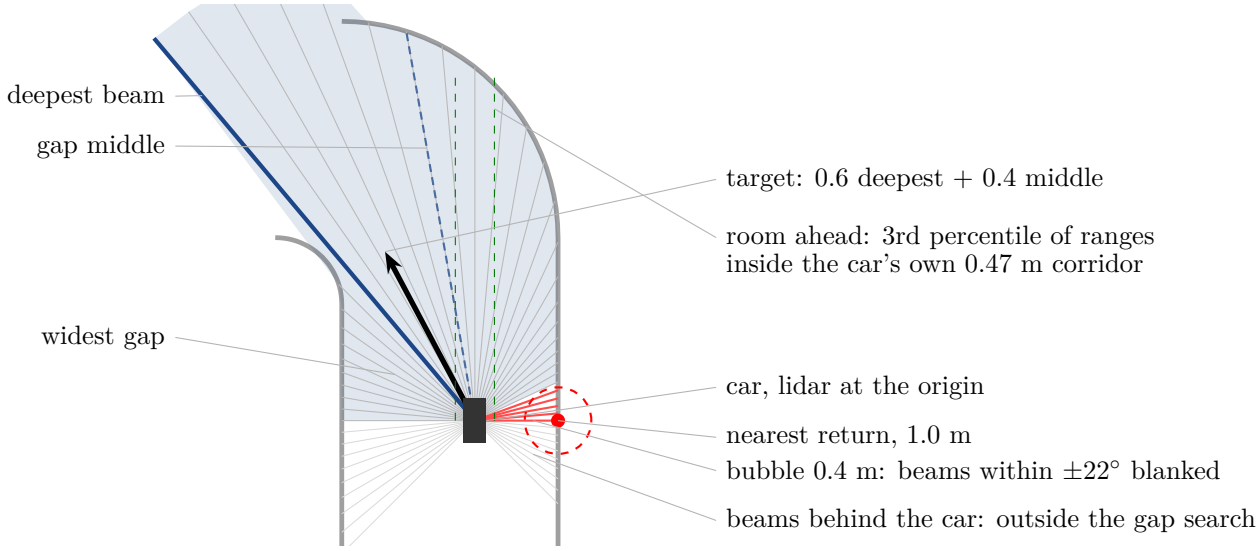
`starter_kit/feb_driver` is a normal ROS 2 Python package. After `cp -r starter_kit/feb_driver stack/` these are the files you own (the Python files live in the inner `feb_driver/` folder):

File	What it is
`package.xml`	The ROS 2 manifest: the package name `feb_driver`, its version and its dependencies (`rclpy`, `sensor_msgs`, `std_msgs`). `colcon` reads it to know the package exists. You only touch it to add a dependency.
`setup.py`	Python packaging. The line that matters is the entry point `reactive_driver = feb_driver.reactive_driver:main`: it is what makes `ros2 run feb_driver reactive_driver` exist. It also installs `launch/` and `config/` so launch files can find them. Add a line here for every new node.
`setup.cfg`	Two lines telling `colcon` where to put the installed scripts. Never edited.
`resource/feb_driver`	An empty marker file ROS 2 uses to index the package. Never edited.
`__init__.py`	Empty; makes the folder a Python module.
`reactive_driver.py`	The driver, about 170 lines, explained below.
`config/driver.yaml`	Every tuning number, one per line with a comment. The launch file hands it to the node, so you change behaviour here without touching code.
`launch/drive.launch.py`	Starts one node, `reactive_driver`, with the yaml; `use_sim_time` follows the box (simulator clock on a laptop, wall clock on the car). This is what `-driver feb_driver` runs.
`driver.yaml`	The driver's card: its name, the kind of track it drives, its modes (`trackdrive`, `inspection`), each a launch file and a speed cap.
`sysid_probe.py`, `launch/sysid.launch.py`	Stage 2. Drives with the same steering but a fixed throttle schedule (three steps, a coast, a chirp) while recording a bag, to measure how the car responds.
`tools/sysid_fit.py`	Fits that bag: steady-state gain (the adapter's `speed_per_throttle`), idle brake, throttle lag and effective wheelbase. Run in the box.

The node subscribes to two topics and publishes one, the real car's: in, `/scan` (`LaserScan`, 1081 ranges over ±135°) and `/odom` (`Odometry`, the car's own speed); out, `/drive` (`AckermannDriveStamped`): a speed in m/s and a steering angle in rad, at most ±0.5236 (30°). Turning the speed into throttle is the sim adapter's job in the simulator and the VESC's on the car.

B How the driver drives, one scan at a time

Every lidar scan, about ten times a second, `on_scan` runs the steps below and publishes one speed and one steering angle. The figure shows one scan from above, in a corridor that bends left; the right wall is the nearest thing the lidar sees.



1. **Clean the scan.** Any beam that returned nothing (infinity) is set to the lidar's maximum range, and every range is clipped to the sensor's limits. This is the numpy array `ranges`; `angles` is the matching bearing of each beam.

2. **Find the nearest return and blank a bubble around it.** Only beams within ±`gap_fov` (1.6 rad, about ±92°) count. The shortest of them is the nearest obstacle, the red dot. Every beam whose bearing is within $\arctan(\texttt{bubble}/r_{\min})$ of that beam is set to zero: at $r_{\min} = 1.0$ m and `bubble`= 0.4 m that is ±22°, the red beams. A zeroed beam can never be chosen as a direction, so the car never aims through a gap that passes within 0.4 m of the closest thing it sees. That is the whole safety idea, and it is why the car brushes walls when `bubble` is too small and refuses corners when it is too big.

3. **Pick the widest gap.** The non-zero beams are split into runs of consecutive beams; the longest run is the gap (blue). Inside it, two candidates: the deepest beam (the longest range) and the middle beam of the run. The target bearing is a blend, `deep_weight` × deepest + (1 − `deep_weight`) × middle, 0.6/0.4 by default. Pure deepest cuts corners tight and hunts; pure middle is smooth but wide.

4. **Turn the bearing into steering.** The car is still turning at its current steering angle for the actuator delay (0.25 s) plus one scan period before the new command takes effect, so the heading change over that time is subtracted from the target first (delay compensation). Then `steer_gain` scales the bearing (0.7: a 20° target asks for 14° of steering), the result is clipped to the ±30° limit, low-passed with `steer_tau` and rate-limited to the actuator's 3.2 rad/s. Without the filtering, a target that jumps from one beam to the next would flick the wheel full lock every scan.

5. **Measure the room ahead.** Take the beams whose end point lies inside the car's own corridor (a strip 0.235 m either side of the centreline, the green dashes). The 3rd percentile of their forward distance is the clearance: not the minimum, which one noisy beam would spoil, but close to it. It is low-passed, except that a genuinely close obstacle (under `brake_margin` +0.3 m) passes straight through so braking is immediate.

6. **Pick a speed.** Three ceilings: `max_speed` (2.5 m/s); the cornering speed $\sqrt{\texttt{lat_accel}/\kappa}$ from the slowly filtered steering, where $\kappa = \tan(\delta)/L$ is the path curvature; and the braking speed $\sqrt{2\,\texttt{decel_limit}\,(\text{clearance} - \texttt{brake_margin})}$, the fastest speed from which the car can still stop before the clearance. The goal is the smallest of the three, floored at `min_speed` (0.9 m/s) so the car never parks in a corner. The goal is low-passed and then slewed at `accel_limit` up and `decel_limit` down.

7. **Publish.** The speed target and the steering angle go out on `/drive`. Under the node, the supervisor clips the speed to the mode's cap and the sim adapter turns it into throttle: feed-forward $v/$`speed_per_throttle` (23 m/s per unit throttle, measured with the sysid tools) plus a small PI trim clipped to half of it, slew-limited, so the command never drops to the simulator's hard-brake zero while speed is wanted. The same law used to live in every driver; now no driver needs it, and on the car the VESC does the job. On shutdown the node publishes speed 0 once so the car stops.

Follow mode (`follow_gap` > 0, used by the house racer as the opponent): if something sits within that distance in a $\pm 15°$ cone straight ahead, it is treated as a car to sit behind, not a wall to steer around. Its beams are replaced by the corridor seen beside it, so the steering keeps the corridor's line, and the speed goal is capped to hold the distance.

Where to start tuning. `bubble` and `deep_weight` decide the line; `max_speed`, `lat_accel` and `brake_margin` decide how fast it dares. Change one number, run the same command, watch the lap time. The sysid tools are for when you want the adapter's throttle law to match the car rather than guess.